



Province de
Hainaut

Institut provincial
Henri La Fontaine



Enseignement
pour Adultes

PROJET D'INTÉGRATION DE DÉVELOPPEMENT

Thème : *SYSTÈME DE GESTION INTERNE POUR SNACK*

Nom : TIEGNI GAMO

Prénom : BERNARD JOEL

Professeur : Mme Passelecq

Année académique 2025-2026

TABLE DES MATIÈRES

RAPPORT TECHNIQUE - SYSTÈME DE GESTION INTERNE POUR SNACK

1. PRÉSENTATION ET ANALYSE FONCTIONNELLE

1.1. Contexte & Problématique

1.2. Objectifs

1.2.1. Objectifs Fonctionnels

1.2.2. Objectifs Techniques

1.3. Public Cible et Valeur Ajoutée

2. CHOIX TECHNOLOGIQUES ET JUSTIFICATION

2.1. Stack Technique

2.2. Justification des Choix

2.3. Environnement de Travail

3. CONCEPTION ET MODÉLISATION

3.1. Modélisation des Données (MCD)

3.1.1. Entités Principales

3.1.2. Schéma Relationnel

3.1.3. Types Personnalisés

3.2. Modèle Physique de Données (MPD)

3.2.1. Script SQL Principal

3.2.2. Contraintes d'Intégrité

3.3. Diagrammes UML

3.3.1. Diagramme des Cas d'Utilisation

UC-01 : Authentification

UC-02 : Gestion des Employés

UC-03 : Gestion des Produits et Stock

UC-04 : Prise de Commande

UC-05 : Encaissement et Clôture

UC-06 : Préparation des Commandes	
UC-07 : Gestion des Tables et Réservations	
UC-08 : Alertes de Stock	
UC-09 : Reporting et Export	
UC-10 : Audit des Opérations	
UC-11 : Commande via QR Code	
3.3.2. Diagramme de Classes	
3.3.3. Diagrammes de Séquence	
Séquence 1 : Prise de Commande Classique	
Séquence 2 : Commande via QR Code	
Séquence 3 : Gestion des Stocks	
3.4. Schéma d'Architecture	
3.4.1. Architecture Globale	
3.4.2. Flux de Données	
4. RÉALISATION TECHNIQUE ET SÉCURITÉ	
4.1. Structure du Code	
4.1.1. Architecture Backend (Spring Boot)	
4.1.2. Architecture Frontend (React)	
4.1.3. Endpoints API Principaux	
Authentification et Gestion Utilisateurs	
Gestion des Commandes	
Système QR Code	
Reporting et Administration	
4.2. Gestion de la Sécurité	
4.2.1. Authentification et Autorisation	
Système JWT	
Gestion des Rôles (RBAC)	
4.2.2. Protection des Données	

Hachage des Mots de Passe (BCrypt)	
Validation des Entrées	
Protection contre les Injections	
4.2.3. Journalisation de Sécurité	
Audit Log	
Monitoring des Accès	
4.3. Interfaces Utilisateur	
4.3.1. Interface Administration	
4.3.2. Interface Serveur/Caissier	
4.3.3. Interface Cuisine	
4.3.4. Interface Client Mobile (QR Code)	
4.3.5. Captures d'Écran	
5. TESTS ET ANALYSE CRITIQUE	
5.1. Tests Réalisés	
5.1.1. Tests Fonctionnels	
Scénarios Métier	
Tests d'Intégration	
5.1.2. Tests Techniques	
Tests de Performance	
Tests de Sécurité	
Tests QR Code	
5.2. Problèmes Rencontrés et Solutions	
5.2.1. Problèmes Techniques	
5.2.2. Solutions Implémentées	
5.2.3. Leçons Apprises	
6. CONCLUSION ET PERSPECTIVES	
6.1. Bilan du Projet	
6.1.1. Objectifs Atteints	

6.1.2. Fonctionnalités Implémentées	
6.2. Améliorations Futures (Perspectives)	
6.2.1. Évolutions Fonctionnelles	
6.2.2. Évolutions Techniques	
6.3. Retour d'Expérience	
6.3.1. Apprentissages Techniques	
6.3.2. Apprentissages Méthodologiques	
ANNEXES	
A. INSTRUCTIONS D'INSTALLATION ET CONFIGURATION ...	
A.1. Prérequis Système	
A.2. Installation Backend (Spring Boot)	
A.3. Installation Frontend (React)	
A.4. Configuration Base de Données	
A.5. Démarrage de l'Application	
B. SCRIPTS SQL COMPLETS	
B.1. Structure Principale des Tables	
B.2. Données de Démonstration	
B.3. Vues et Fonctions Utilitaires	
C. CONFIGURATIONS TECHNIQUES	
C.1. Configuration Spring Boot	
C.2. Configuration React	
C.3. Variables d'Environnement	
D. DÉPÔT GIT ET DOCUMENTATION	
D.1. Structure du Dépôt	
D.2. Documentation API	
D.3. Guide de Contribution	
E. RÉFÉRENCES TECHNIQUES	
E.1. Bibliographie	

E.2. Liens Utiles

GLOSSAIRE

INDEX

1. PRÉSENTATION ET ANALYSE FONCTIONNELLE

1.1. Contexte & Problématique

Le snack rencontre des difficultés dans la gestion de ses activités quotidiennes : prise de commandes manuelle, suivi des stocks approximatif, absence de centralisation des données entre les différents acteurs (serveurs, cuisiniers, caissiers). Cette fragmentation entraîne des erreurs, des pertes de temps et une traçabilité limitée des opérations.

Problématique : Comment centraliser et optimiser la gestion des commandes, du stock, des réservations et du personnel dans un snack à travers une application web complète ?

1.2. Objectifs

Fonctionnels :

- ✓ Permettre la prise de commandes multi-produits (sur place/à emporter)
- ✓ Gérer le stock en temps réel avec alertes automatiques
- ✓ Administrer les réservations de tables
- ✓ Générer des tickets de caisse
- ✓ Gérer les rôles et permissions (Admin, Serveur, Cuisinier, Caissier)
- ✓ Suivre les ventes et générer des rapports
- ✓ Permettre aux clients de laisser des avis
- ✓ Synchroniser en temps réel les commandes clients avec l'interface serveur
- ✓ Chatbot pour le service client permettant aux utilisateurs d'avoir plus de renseignement sur l'application
- ✓ Rendre l'application responsive pour s'adapter à différent écran (mobile et pc)
- ✓ Gestion des paiements pour permettre aux utilisateurs de finalisée leur commande

Techniques :

- ✓ Développer une application web full-stack
- ✓ Mettre en œuvre une architecture MVC avec Spring Boot et React
- ✓ Assurer la sécurité des données (authentification, hachage)
- ✓ Utiliser PostgreSQL comme SGBD relationnel

- ✓ Implémenter une API REST sécurisée
- ✓ Mettre en place un système d'authentification client simplifié
- ✓ Utilisation d'un modèle IA (intelligence artificielle) optimal pour le chatbot

1.3. Public Cible et Valeur Ajoutée

Public : Propriétaire/gérant du snack, personnel (serveurs, cuisiniers, caissiers)

Valeur ajoutée :

- 😊 Gain de temps sur les opérations quotidiennes
- 😊 Réduction des erreurs et des pertes
- 😊 Meilleure traçabilité des opérations
- 😊 Optimisation de la gestion des stocks
- 😊 Amélioration de l'expérience client
- 😊 Chatbot pour le service client accessible 24h/24
- 😊 Application responsive et adaptative pour différent écran

2. CHOIX TECHNOLOGIQUES ET JUSTIFICATION

2.1. Stack Technique

- 😊 Backend: Spring Boot (REST API) avec Spring Data JPA, Spring Security
- 😊 Base de données : PostgreSQL (version 14/15+)
- 😊 Frontend : React (Single Page Application)
- 😊 Gestion des migrations : Flyway ou Liquibase
- 😊 Authentification : JWT (JSON Web Tokens)
- 😊 Hachage mots de passe : BCrypt
- 😊 Outils développement : Java JDK 17/21, Maven, Node.js/npm
- 😊 IDE: Visual Studio Code
- 😊 Gestion de version : Git
- 😊 Administration BD : pgAdmin
- 😊 Modèle IA : openAI, Gemini
- 😊 Machine virtuelle pour l'hébergement : Microsoft AZURE

2.2. Justification des Choix

- 😊 Spring Boot : Framework mature avec conventions facilitant le développement d'API REST sécurisées. Intégration performante avec JPA/Hibernate et Spring Security.

- ☺ PostgreSQL : SGBD relationnel robuste, conforme ACID, offrant des fonctionnalités avancées (transactions, types, fonctions stockées) adaptées aux besoins métiers complexes.
- ☺ React : Permet un développement rapide d'interfaces riches et réactives. Large communauté et facilité d'intégration avec des API REST. Possibilités d'évolution vers une PWA (Progressive Web App).
- ☺ Architecture REST : Standard pour les applications web modernes, favorise la séparation des responsabilités et la scalabilité.
- ☺ Sécurité : Combinaison de BCrypt pour le hachage sécurisé des mots de passe, JWT pour la gestion des sessions stateless, et RBAC (Role-Based Access Control) via Spring Security.
- ☺ Modèle IA(OpenAI ou Gemini) : plus optimale pour le traitement du langage naturel facilitant la mise en place des assistantes virtuelle plus professionnelle
- ☺ Microsoft Azure offre flexibilité, sécurité,scalabilité et intégration native avec l'écosystème Microsoft, tout en permettant des économies significatives sur l'infrastructure IT

2.3. Environnement de Travail

- Système d'exploitation : Windows 10/11
- Configuration requise : Accès administrateur pour les installations

Outils installés :

- ✓ Java JDK 17+
- ✓ Apache Maven
- ✓ PostgreSQL + pgAdmin
- ✓ Node.js 16+ et npm
- ✓ Git

Environnement de développement : Configuration locale sans Docker pour simplification

3. CONCEPTION ET MODÉLISATION

3.1. Modélisation des Données (MCD)

Entités principales identifiées :

Role : Définit les permissions globales (ADMIN, CAISSIER, SERVEUR, CUISINIER)

Employee : Informations du personnel (nom, prénom, contact, salaire, etc.)

User : Compte d'accès au système (identifiants, mot de passe haché, rôle)

Product : Produits vendus (nom, prix, quantité disponible, seuil d'alerte)

TableSnack : Tables physiques du snack (numéro, capacité, statut)

Sale : En-tête de vente (montant total, mode paiement, type commande, statut)

SaleItem : Lignes de vente (produit, quantité, prix unitaire, total ligne)

StockMovement : Historique des mouvements de stock

AlertStock : Alertes de réapprovisionnement

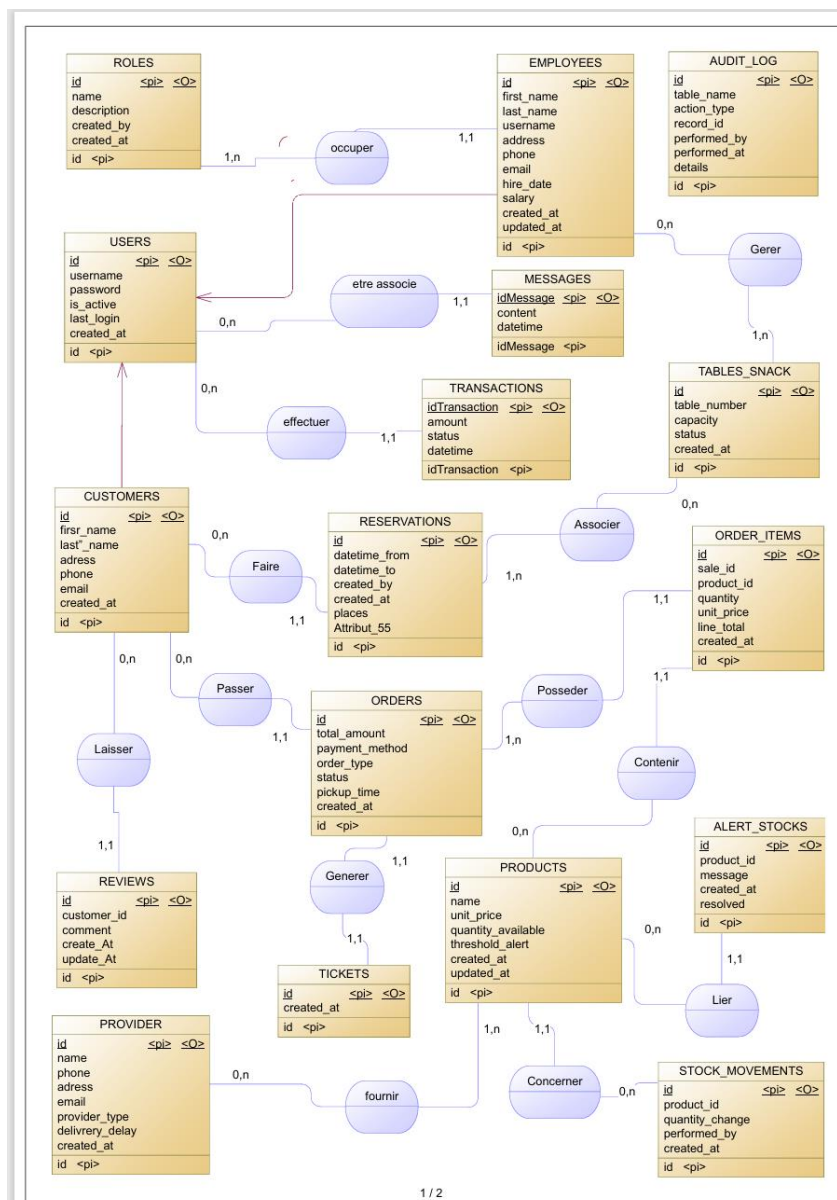
AuditLog : Journal d'audit de toutes les opérations critiques

Reservation : Réservations de tables

Ticket : Tickets de caisse générés

Message : Message envoyé par le client et message reçu par le client ou chatbot

Transaction : Transaction de paiement ou remboursement (toute transaction d'argent)



Dictionnaire de données - Gestion Snack

Voici un modèle de dictionnaire de données pour mon application Spring Boot avec toutes les tables et attributs :

1. Table users (Utilisateurs)

Attribut	Type	Taille	Nullable	Clé	Description
user_id	BIGINT	-	NON	PK	Identifiant unique
username	VARCHAR	50	NON	UK	Nom d'utilisateur
email	VARCHAR	100	NON	UK	Email

password_hash	VARCHAR	255	NON	-	Mot de passe crypté
first_name	VARCHAR	50	OUI	-	Prénom
last_name	VARCHAR	50	OUI	-	Nom
phone	VARCHAR	20	OUI	-	Téléphone
role_id	BIGINT	-	NON	FK	Rôle utilisateur
is_active	BOOLEAN	-	NON	-	Compte actif
created_at	TIMESTAMP	-	NON	-	Date création
updated_at	TIMESTAMP	-	OUI	-	Date modification

2. Table roles (Rôles)

Attribut	Type	Taille	Nullable	Clé	Description
role_id	BIGINT	-	NON	PK	Identifiant
role_name	VARCHAR	50	NON	UK	ADMIN, EMPLOYEE, CUSTOMER, etc.
description	VARCHAR	255	OUI	-	Description

3. Table customers (Clients)

Attribut	Type	Taille	Nullable	Clé	Description
customer_id	BIGINT	-	NON	PK	Identifiant
user_id	BIGINT	-	NON	FK	Référence utilisateur
loyalty_points	INTEGER	-	OUI	-	Points fidélité
preferred_payment	VARCHAR	50	OUI	-	Mode paiement préféré
allergy_info	TEXT	-	OUI	-	Informations allergies

4. Table products (Produits)

Attribut	Type	Taille	Nullable	Clé	Description
product_id	BIGINT	-	NON	PK	Identifiant
name	VARCHAR	100	NON	-	Nom produit
category	VARCHAR	50	NON	-	Catégorie (boisson, sandwich, dessert)
price	DECIMAL	10,2	NON	-	Prix
stock_quantity	INTEGER	-	NON	-	Quantité stock
is_available	BOOLEAN	-	NON	-	Disponible à la vente

3.2. Diagrammes UML

3.2.1. Diagramme des Cas d'Utilisation

Acteurs principaux :

Administrateur : Accès complet au système

Caissier : Encaissement, génération tickets

Serveur : Prise de commandes, gestion tables

Cuisinier : Préparation commandes

Système (automate) : Tâches automatiques

Cas d'utilisation documentés (UC-01 à UC-11) :

UC-01 : Authentification

UC-02 : Gestion des employés

UC-03 : Gestion des produits et du stock

UC-04 : Prise de commande (sur place/à emporter)

UC-05 : Encaissement et clôture de vente

UC-06 : Préparation des commandes (cuisine)

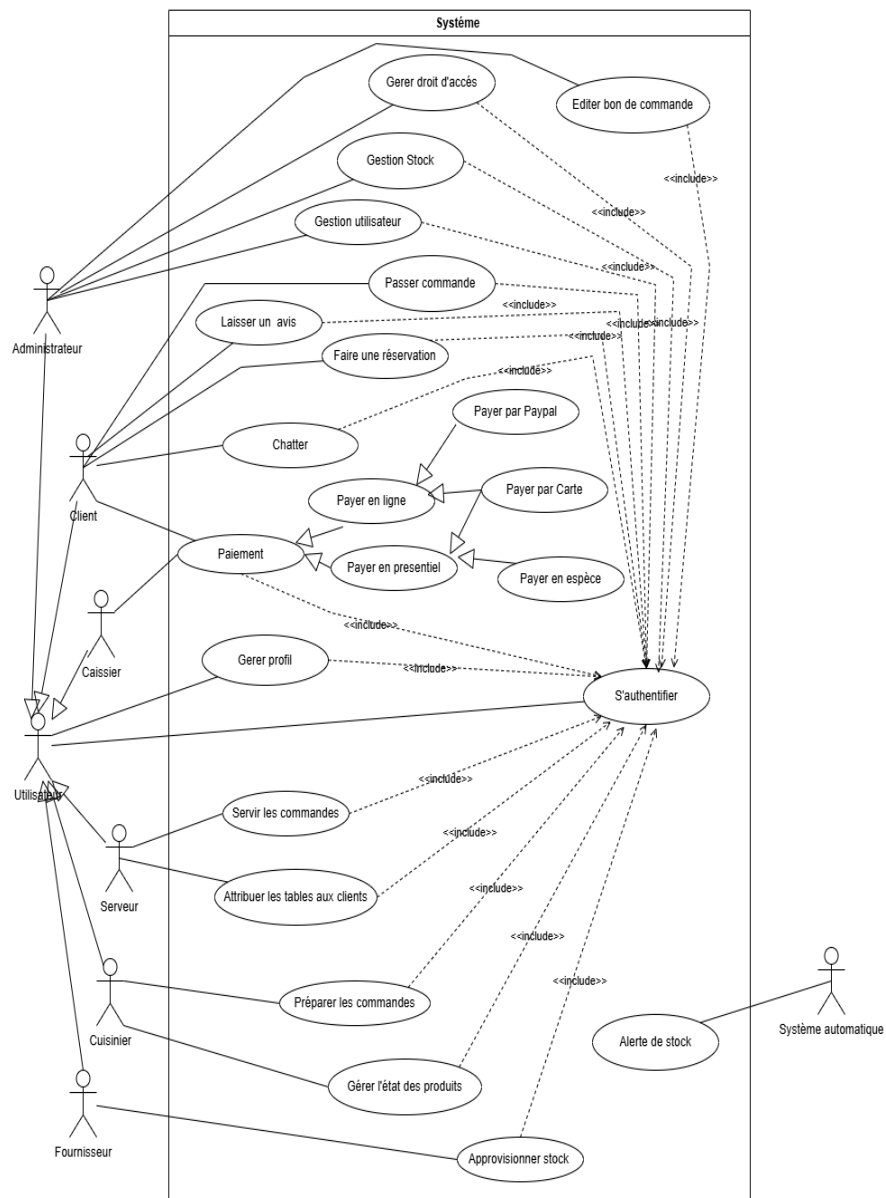
UC-07 : Gestion des tables et réservations

UC-08 : Alertes de stock et réapprovisionnement

UC-09 : Reporting et export

UC-10 : Audit des opérations

UC-11 - Commande via QR Code



3.2.2. Diagramme de Classes

(Basé sur les entités décrites dans la modélisation)

Classes principales avec attributs :

Role : id, name, description, created_at

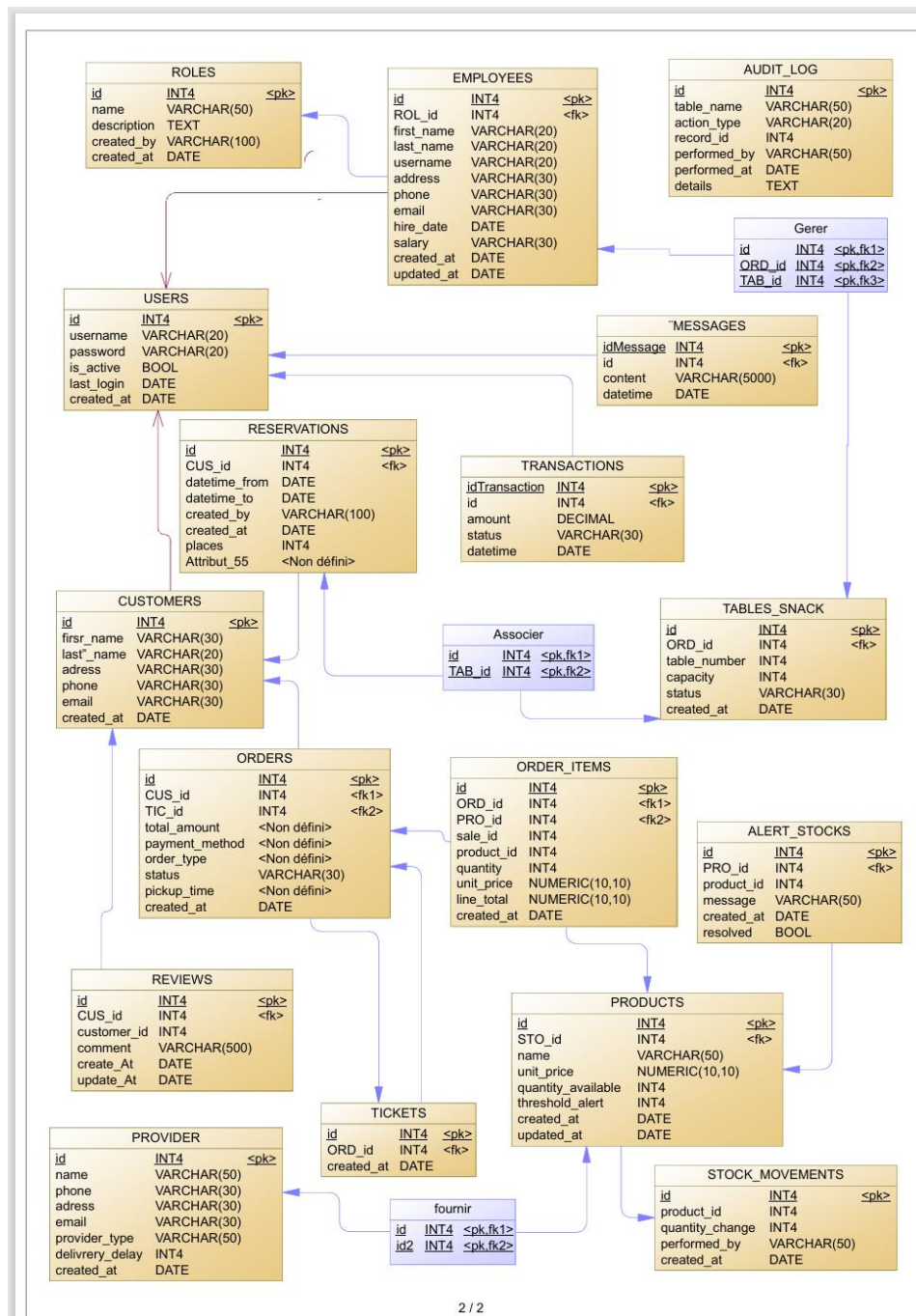
Employee : id, first_name, last_name, username, address, phone, email, hire_date, salary, role_id, created_at, updated_at

User : id, username, password_hash, employee_id, role_id, is_active, last_login, created_at

Product : id, name, unit_price, quantity_available, threshold_alert, created_at, updated_at

Sale : id, employee_id, table_id, total_amount, payment_method, order_type, status, pickup_time, created_at

SaleItem : id, sale_id, product_id, quantity, unit_price, line_total, created_at



3.2.3. Diagramme de Séquence - Exemple : Prise de Commande

text

1. Serveur → Système : Création nouvelle vente
2. Système → Base de données : Vérification disponibilité produits
3. Base de données → Système : Retour des stocks disponibles
4. Système → Serveur : Affichage des produits disponibles
5. Serveur → Système : Ajout des lignes de vente
6. Système → Base de données : Enregistrement vente et mise à jour stock

7. Système → Cuisinier : Notification nouvelle commande

8. Système → Serveur : Confirmation et génération ticket provisoire

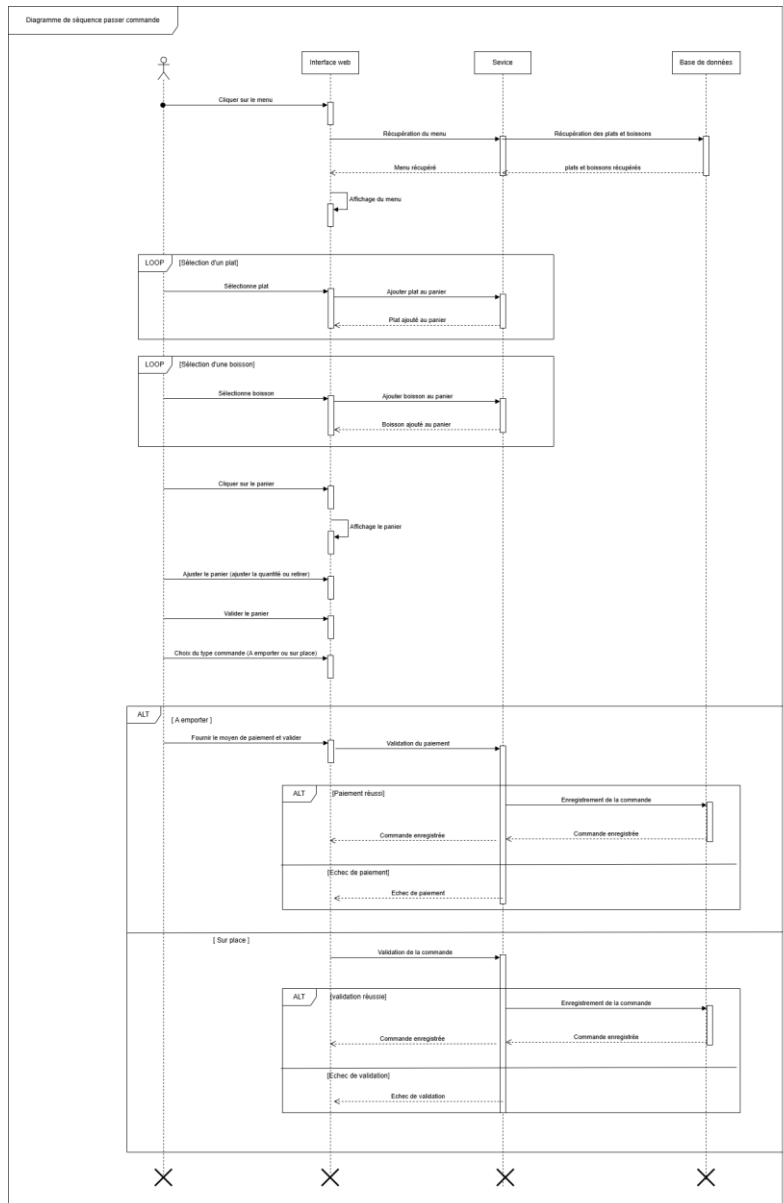


diagramme séquence de chatter

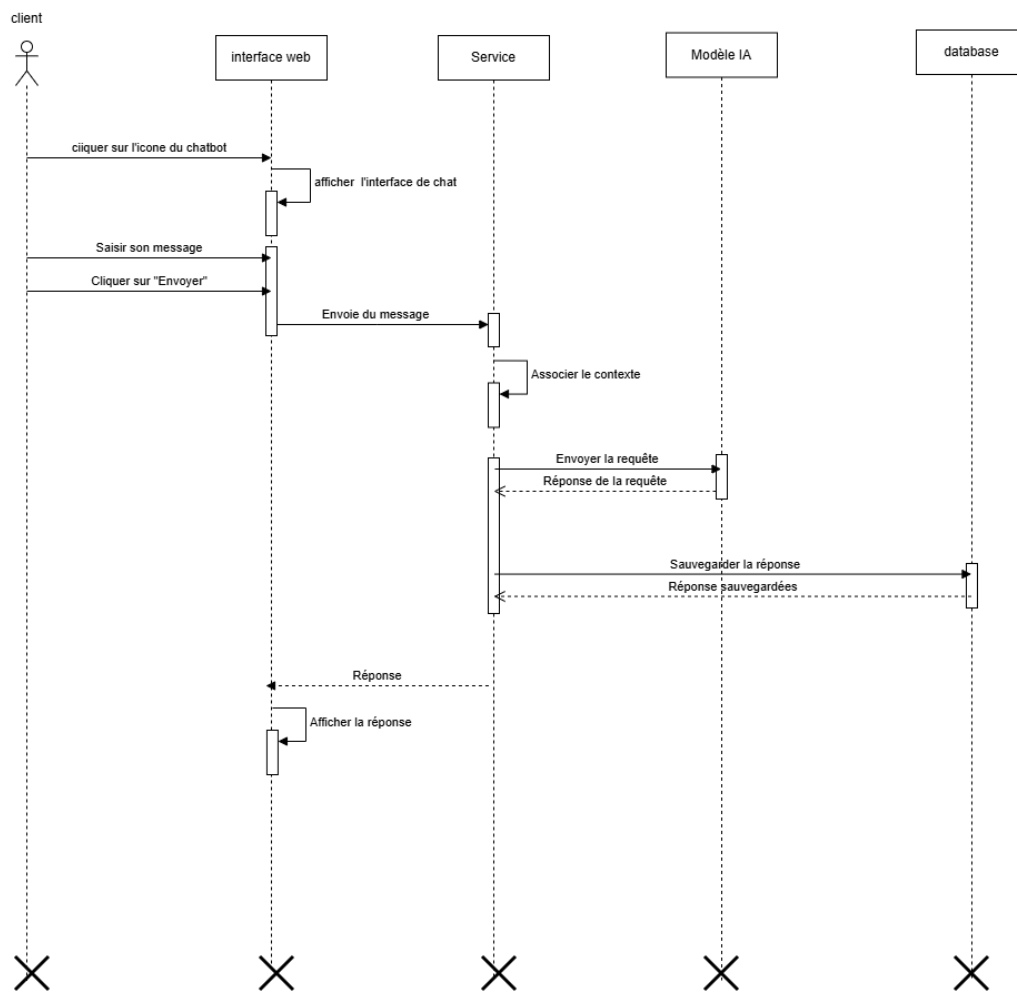
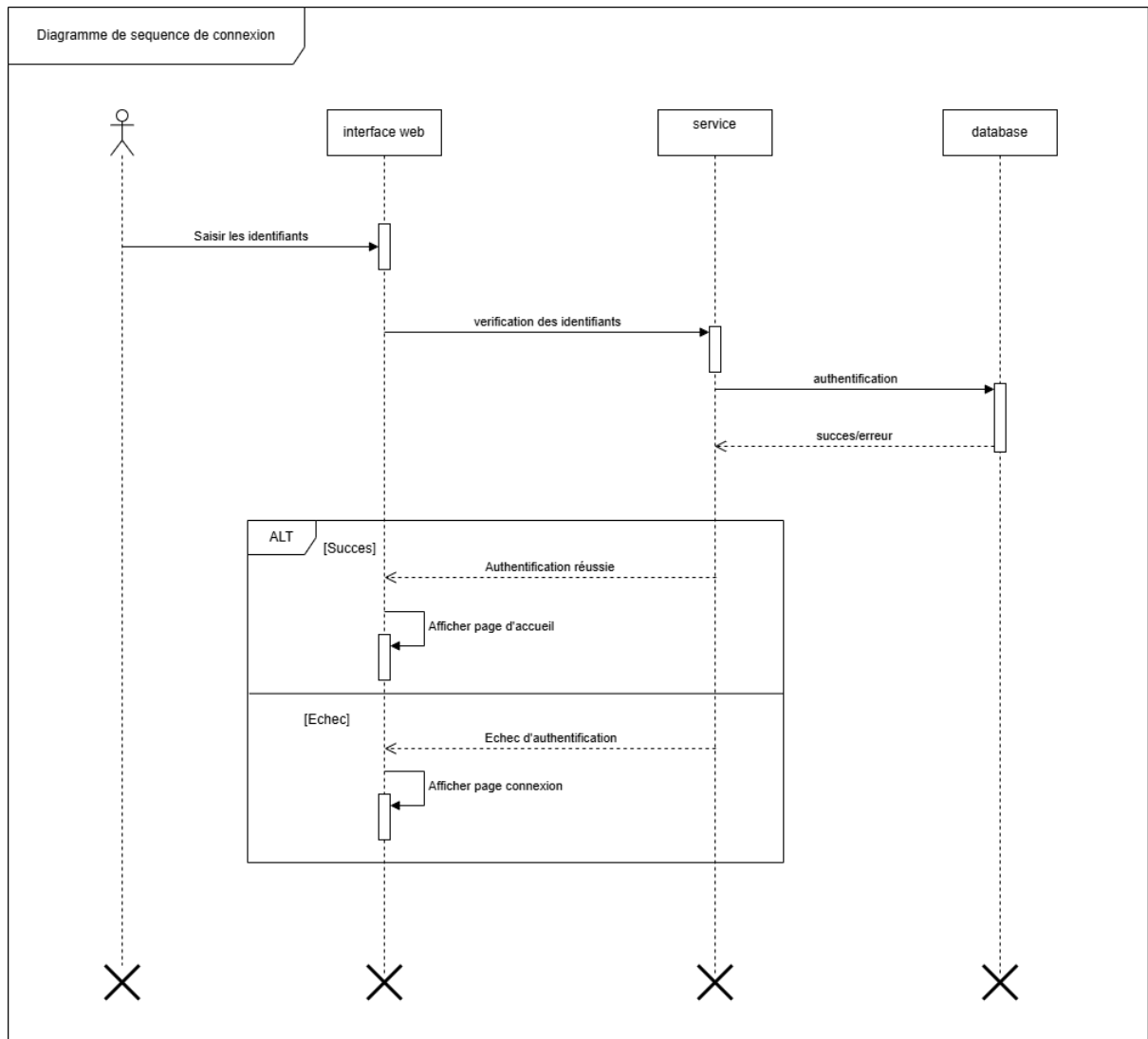
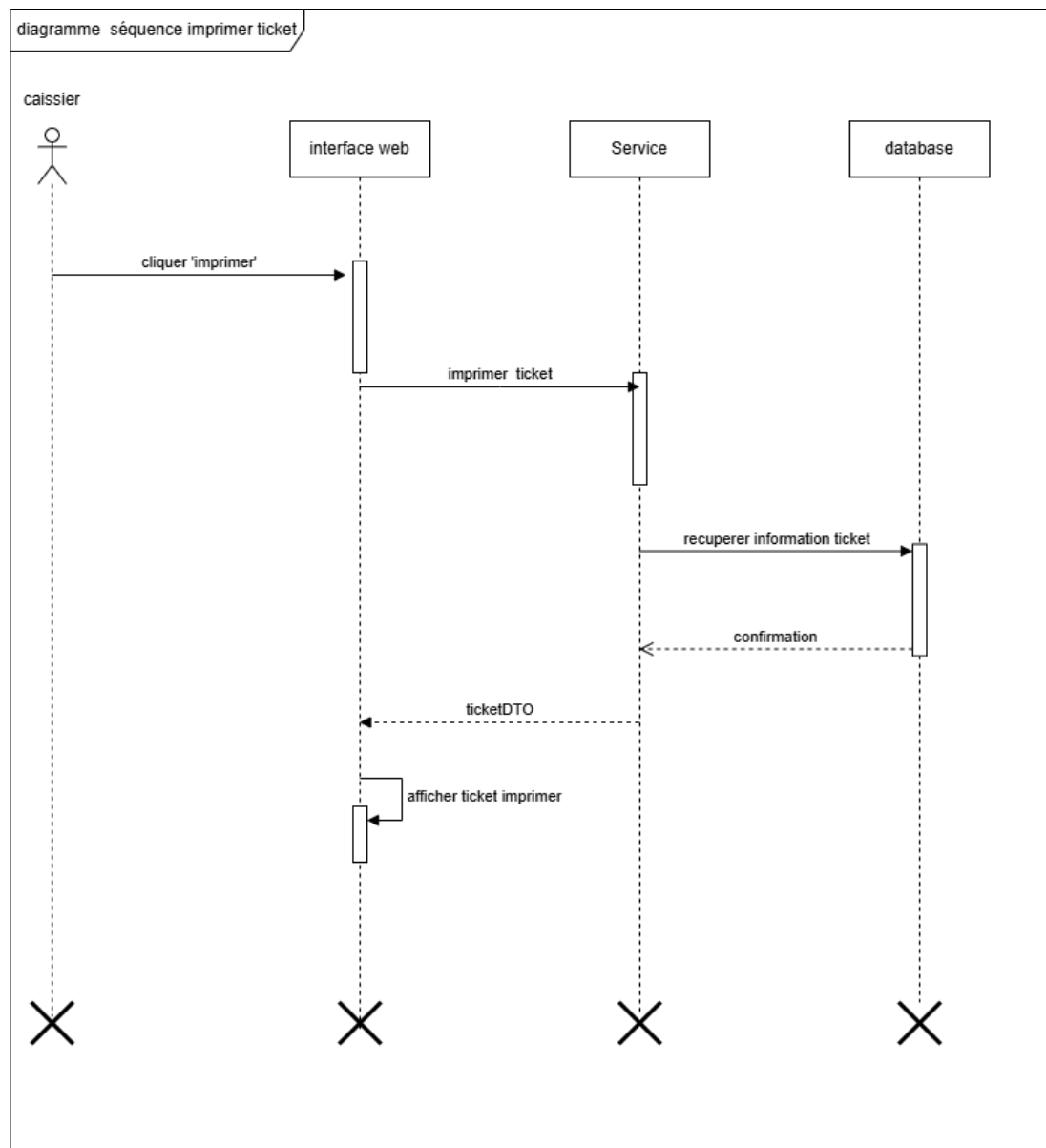
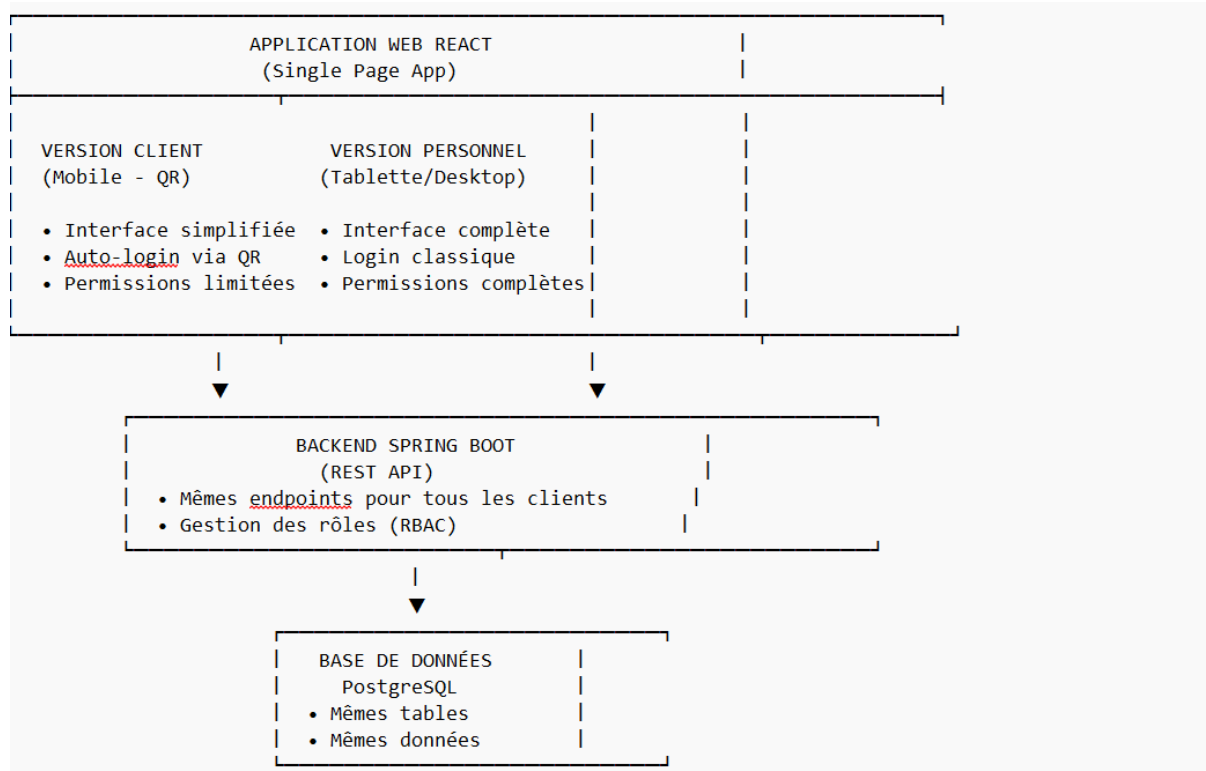


Diagramme de sequence de connexion





3.3. Schéma d'Architecture



4. RÉALISATION TECHNIQUE ET SÉCURITÉ

4.1. Structure du Code

Architecture backend (Spring Boot) :

- Controllers : Gestion des endpoints REST (@RestController)
- Services : Logique métier (@Service)
- Repositories : Accès aux données via Spring Data JPA (@Repository)
- Entities : Mapping objet-relationnel (@Entity)
- Security : Configuration Spring Security (@Configuration)
- DTOs : Data Transfer Objects pour les échanges API

Principaux endpoints API :

- POST /api/auth/login - Authentification (retourne JWT)
- GET /api/products - Liste des produits
- POST /api/sales - Création d'une vente
- PUT /api/sales/{id}/close - Clôture d'une vente
- GET /api/reports/sales - Génération de rapports
- GET /api/audit-logs - Consultation du journal d'audit

4.2. Gestion de la Sécurité

Mesures implémentées :

Authentification :

- Formulaire identifiant/mot de passe
- Génération de token JWT après succès
- Stockage sécurisé des mots de passe avec BCrypt

Contrôle d'accès (RBAC) :

- Rôles définis : ADMIN, CAISSIER, SERVEUR, CUISINIER
- Autorisations par endpoint via @PreAuthorize

Exemple : @PreAuthorize("hasRole('ADMIN') or hasRole('CAISSIER')")

Journalisation de sécurité :

- Toutes les authentifications enregistrées dans AuditLog
- Actions sensibles (CRUD) tracées automatiquement
- Tentatives d'accès non autorisées loggées

Protection des données :

- Validation des entrées utilisateur
- Communication HTTPS en production
- Protection contre les injections SQL via JPA

Configuration Spring Security :

java

@Configuration

@EnableWebSecurity

public class SecurityConfig {

 @Bean

 public PasswordEncoder passwordEncoder() {

 return new BCryptPasswordEncoder();

 }

 // Configuration des règles d'accès

 // Filtre JWT

 // Gestion des CORS

}

4.3. Interfaces Utilisateur

Pages principales développées :

Page de connexion - Authentification des utilisateurs

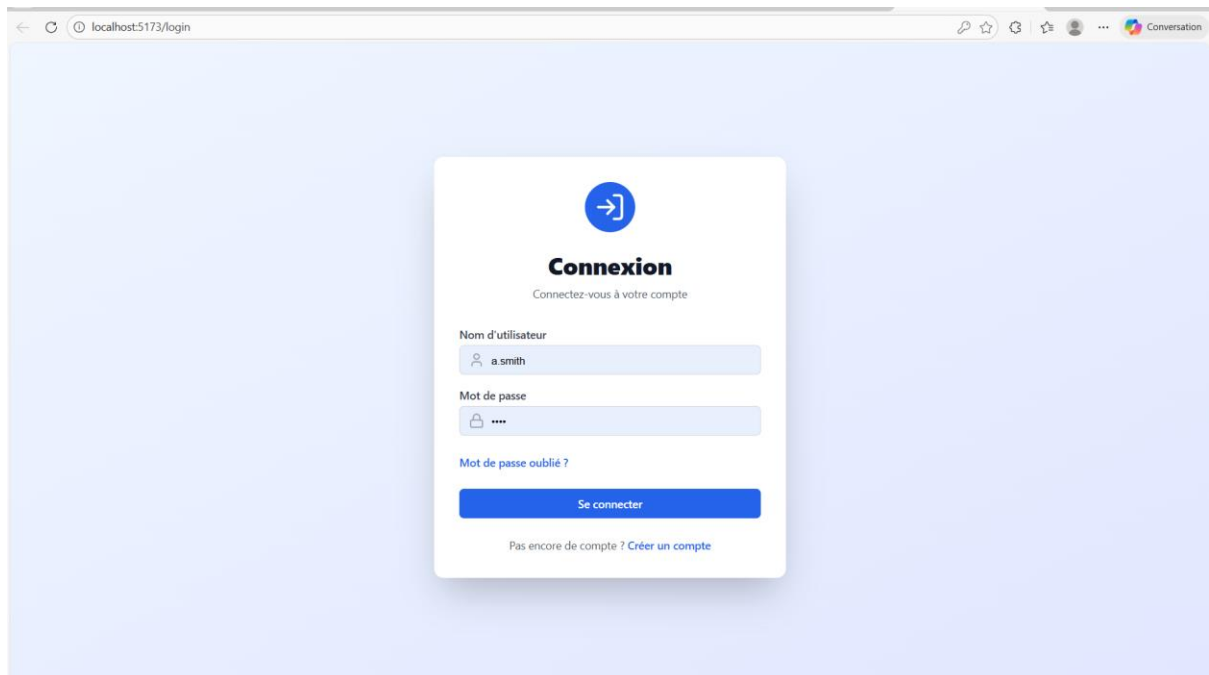
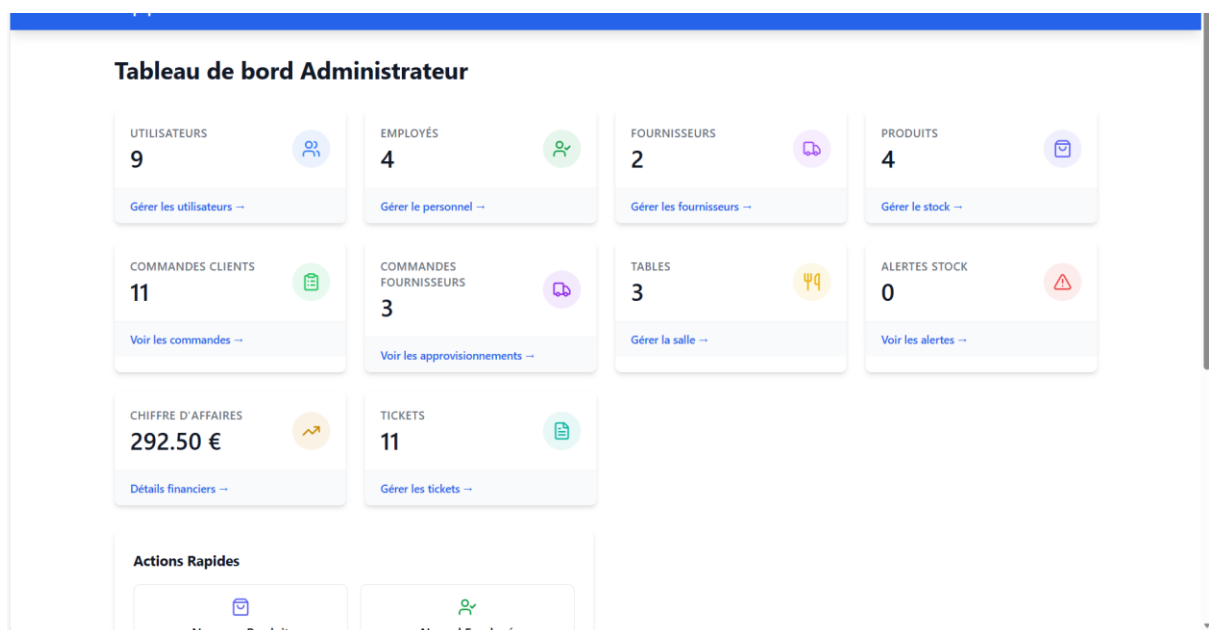


Tableau de bord - Vue d'ensemble selon le rôle



Gestion des commandes - Interface de prise de commande

Gestion des Commandes

Tous les statuts

ID	CLIENT	DATE	TOTAL	STATUT	ACTIONS
#11	2 (Marie Simo)	12/12/2025 01:00:00	8.5 €	Active	

Gestion des stocks - Liste produits avec alertes visuelles

Gestion des produits

Gérer tous les produits du snack

Tous les types

Sandwich

Nourriture

Prix unitaire: 5.50 €

Stock disponible: 93

Seuil d'alerte: 10

Modifier

Pizza

Nourriture

Prix unitaire: 12.00 €

Stock disponible: 245

Seuil d'alerte: 5

Modifier

Durum Pitta

Nourriture

Prix unitaire: 7.00 €

Stock disponible: 44

Seuil d'alerte: 30

Modifier

Soda

Boisson

Prix unitaire: 1.50 €

Stock disponible: 692

Seuil d'alerte: 20

Modifier

Gestion des tables - Plan interactif des tables

Gestion des Tables

Dernière mise à jour: 14:11:21

#1

Capacité 4 pers.

FREE

Libérer

Occuper

Réserver

#2

Capacité 2 pers.

RESERVED

Libérer

Occuper

Réserver

#3

Capacité 6 pers.

FREE

Libérer

Occuper

Réserver

Reporting - Graphiques et export de données

Caractéristiques de l'interface :

Responsive design (adapté aux tablettes utilisées en salle)

Navigation intuitive par rôle

Génération de tickets imprimables

4.3.1. Interface QR Code Client

L'interface client via QR code permet aux clients de commander directement depuis leur smartphone en scannant le code présent sur leur table.

L'interface client via QR code permet aux clients de commander directement depuis leur smartphone en scannant le code présent sur leur table.

- ✓ Accès via QR code : Chaque table dispose d'un QR code unique généré via <https://genqr.com/>
- ✓ Lien de redirection : [http://localhost:5173/customer/menu?table=\[ID_TABLE\]](http://localhost:5173/customer/menu?table=[ID_TABLE]) (version développement)



- ✓ Interface responsive : Adaptation automatique aux smartphones
- ✓ Processus simplifié :

1-Scan du QR code

2-Affichage du menu adapté

3-Sélection des produits

4-Validation de la commande

5-Notification automatique en cuisine

Captures d'écran (QR code généré et l'interface mobile correspondante):



Note technique :

Pour la démonstration académique, l'application tournant en local (localhost), les QR codes ne sont accessibles que depuis la machine de développement. En production, l'hébergement cloud permettrait un accès public via un domaine dédié.

5. TESTS ET ANALYSE CRITIQUE

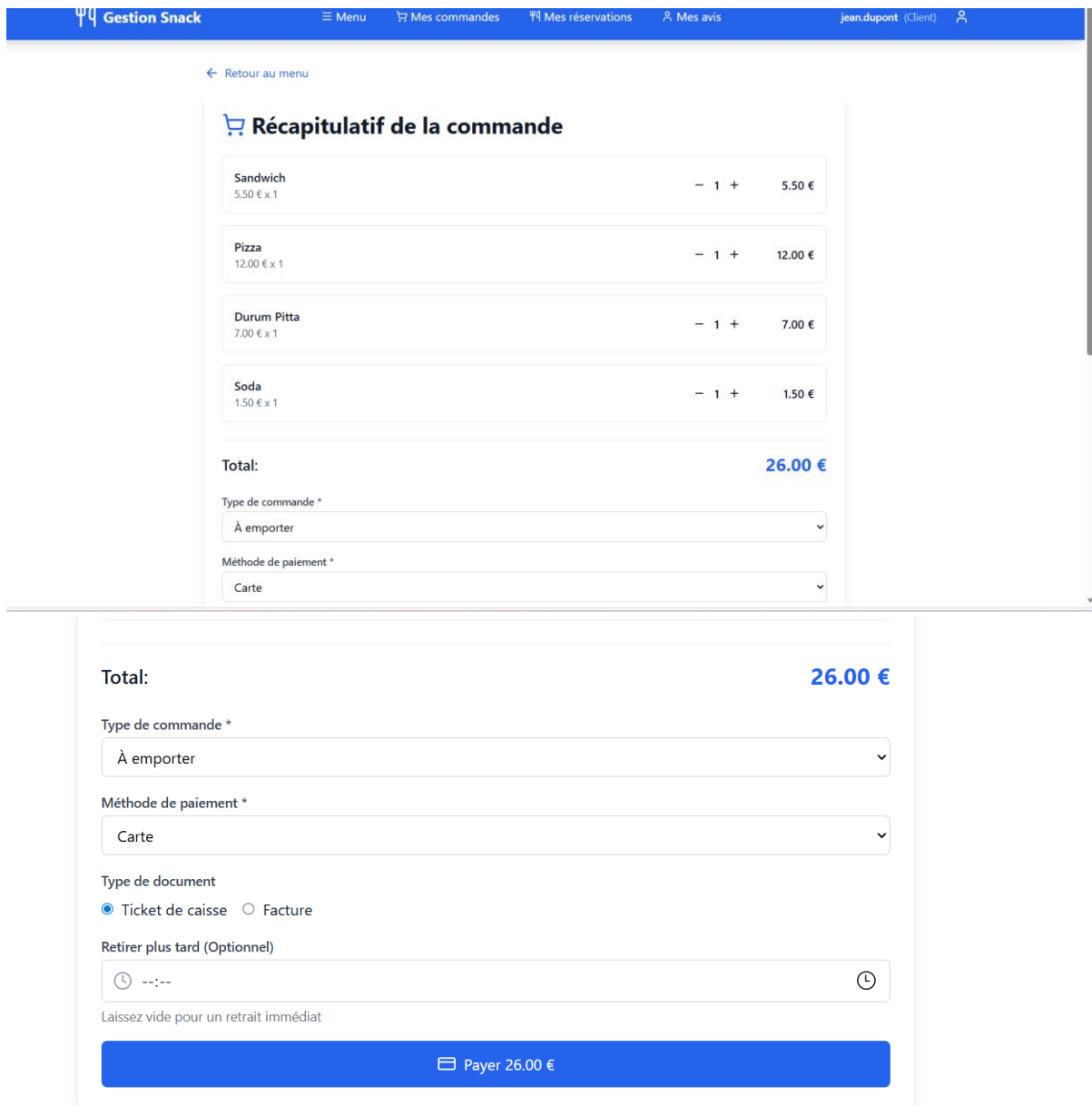
5.1. Tests Réalisés

Scénarios de test fonctionnels validés :

Authentification : Connexion avec différents rôles



Prise de commande : Création vente multi-produits



Gestion stock : Déclenchement alerte seuil bas

Après qu'un client a passé la commande suivante

Commande #12
Date: 12/12/2025

Articles:
- Durum Pitta x 32 = 224.00 €

Type: À emporter
 Paiement: Carte
Total: **224.00 €**

× Annuler

🖨️ Imprimer ticket

🛒 Payer

le seuil d'alerte se déclenche comme suit

Gestion Snack
Tableau de bord Utilisateurs Produits Commandes a.smith (Administrateur)

Alertes de Stock
Suivi des stocks faibles Actualiser

⚠️

Produit ID: 5
⚠️ Alerte stock faible : Durum Pitta - 12 unités restantes.
Détecté le: 12/12/2025 14:20:51

✅ Marquer comme résolu

Encaissement : Clôture vente et génération ticket

Réservation : Gestion des conflits de créneaux

Reporting : téléchargement PDF/CSV des ventes

Tests techniques :

Tests de connexion à la base de données PostgreSQL

Validation des contraintes d'intégrité (clés étrangères)

Tests de performance avec volumes de données réalistes

Tests de sécurité (validation des autorisations par rôle)

```
mpl.java | application.properties Backend_gestion_snack\... | AuthContext.jsx | snack_db_postgres.sql Backend_gestion_snack
Backend_gestion_snack > src > main > java > com > joel > gestion_snack > config > WebConfig.java > WebConfig
1 package com.joel.gestion_snack.config;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.web.servlet.config.annotation.CorsRegistry;
6 import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
7
8 @Configuration
9 public class WebConfig implements WebMvcConfigurer {
10
11     @Override
12     public void addCorsMappings(CorsRegistry registry) {
13         registry.addMapping("/**") // Permettre CORS pour toutes les routes
14             .allowedOrigins("http://localhost:5173") // Autoriser votre origine
15             .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS") // Méthodes autorisées
16             .allowedHeaders("*"); // Autoriser tous les en-têtes
17     }
18 }
```

5.2. Problèmes Rencontrés et Solutions

Problème : Gestion des transactions concurrentes sur le stock lors de commandes simultanées

Solution : Utilisation du niveau d'isolation REPEATABLE_READ et verrouillage optimiste

Problème : Performances des requêtes de reporting sur de grandes périodes

Solution : Indexation stratégique, pagination des résultats, agrégations pré-calculées

Problème : Gestion des timezones pour les réservations et horaires

Solution : Standardisation sur UTC en base de données, conversion locale côté frontend

Problème : Sérialisation cyclique des entités JPA dans les réponses JSON

Solution : Utilisation de DTOs spécifiques par use case

Problème : Déploiement des migrations de base de données

Solution : Implémentation de Flyway pour une gestion versionnée et reproductible

6. CONCLUSION ET PERSPECTIVES

6.1. Bilan du Projet

Objectifs atteints :

- ✓ Centralisation complète des processus métier
- ✓ Gestion fine des rôles et permissions (RBAC)

- ✓ Système de stock avec alertes automatiques
- ✓ Journal d'audit pour traçabilité complète
- ✓ Interface web responsive et intuitive
- ✓ Architecture technique robuste et maintenable

Fonctionnalités clés implémentées :

- ✓ Authentification sécurisée multi-rôles avec JWT
- ✓ Prise de commandes multi-produits (sur place/à emporter)
- ✓ Gestion des réservations de tables
- ✓ Alertes de stock automatiques
- ✓ Génération de tickets et rapports exportables
- ✓ Interface administrateur complète
- ✓ Système QR code complet (démonstration locale) - Permet aux clients de commander via leur smartphone en scannant un code sur la table. L'interface mobile responsive est entièrement fonctionnelle, avec limitation d'accès due à l'hébergement local pour la démonstration académique.

6.2. Améliorations Futures (Perspectives)

- Application mobile - Version React Native pour les serveurs en salle
- Paiements en ligne - Intégration d'API de paiement (Stripe, PayPal)
- Notifications en temps réel - WebSocket pour mise à jour immédiate cuisine/salle
- Business Intelligence avancée - Tableaux de bord prédictifs, analyse des tendances
- Gestion des recettes et coûts - Calcul automatique des marges par produit
- Intégration matérielle - Connexion avec terminaux de paiement et imprimantes tickets
- Module de fidélité client - Programme de points et promotions
- Version cloud - Déploiement sur AWS/Azure avec haute disponibilité
- Génération automatique des QR codes via une API dédiée
- QR codes dynamiques avec tokens d'authentification temporaires
- Intégration avec Google/Apple Wallet pour les cartes de fidélité
- Analytics : Suivi des scans et conversion commande/scan
- Hébergement cloud pour le système QR code : Déploiement sur un serveur public (AWS, Azure, Heroku) avec domaine dédié (snack-commande.com) pour rendre les QR codes accessibles depuis n'importe quel smartphone. Intégration HTTPS et génération automatique des QR codes via API.
- Génération automatique de QR codes : Implémentation d'un endpoint API `/api/tables/{id}/qrcode` qui génère dynamiquement les QR codes avec tokens sécurisés temporaires.

- Application mobile native : Développement d'une application React Native pour améliorer l'expérience utilisateur sur smartphone (notifications push, mode hors-ligne, intégration caméra native).
- Système de paiement intégré : Connexion avec Stripe ou PayPal permettant aux clients de payer directement depuis leur interface mobile après scan du QR code.
- Analytics des scans : Tableau de bord montrant les statistiques d'utilisation des QR codes (nombre de scans par table, conversion en commandes, heures de pointe).

6.3. Retour d'Expérience

Ce projet a permis de mettre en pratique une méthodologie complète de développement logiciel, depuis l'analyse des besoins jusqu'au déploiement. L'utilisation combinée de Spring Boot et React a démontré sa pertinence pour développer des applications web d'entreprise robustes et maintenables. La modélisation soignée de la base de données PostgreSQL a été cruciale pour garantir l'intégrité des données et les performances du système.

Les principaux apprentissages techniques incluent la mise en œuvre d'une sécurité robuste (JWT, BCrypt, RBAC), la gestion des transactions complexes, et le développement d'une API REST bien structurée. Sur le plan méthodologique, l'importance d'une documentation précise et d'une modélisation rigoureuse a été confirmée comme facteur clé de succès.

ANNEXES

A. Instructions d'Installation et Configuration

Prérequis :

- 😊 Windows 10/11 avec accès administrateur
- 😊 Java JDK 17+ (Eclipse Temurin recommandé)
- 😊 Apache Maven 3.6+
- 😊 PostgreSQL 14/15+ avec pgAdmin
- 😊 Node.js 16+ et npm
- 😊 Git

Étapes d'installation :

- Configurer Java : Définir JAVA_HOME et ajouter au PATH
- Configurer Maven : Définir M2_HOME et ajouter au PATH

- Installer PostgreSQL : Créer la base gestion_snack et l'utilisateur snack_user
- Exécuter le script SQL : Importer la structure via pgAdmin ou psql
- Configurer le backend : Modifier application.properties avec les identifiants BD
- Lancer le backend : mvn spring-boot:run depuis le dossier du projet
- Configurer le frontend : npm install puis npm start depuis le dossier React
- Accéder à l'application : <http://localhost:5173/>

B. Scripts SQL (Extrait principal)

-- Create enum types

```
CREATE TYPE role_type AS ENUM
('ADMIN','CUSTOMER','CASHIER','WAITER','COOK','PROVIDER');

CREATE TYPE product_type AS ENUM ('FOOD','DRINK');

CREATE TYPE table_status_type AS ENUM ('FREE','OCCUPIED','RESERVED');

CREATE TYPE payment_method_type AS ENUM ('CASH','CARD');

CREATE TYPE order_type_type AS ENUM ('ON_SITE','TAKEAWAY');

CREATE TYPE order_status_type AS ENUM ('ACTIVE','CLOSED','CANCELLED');

CREATE TYPE reservation_status_type AS ENUM
('BOOKED','CANCELLED','COMPLETED');
```

-- Roles

```
CREATE TABLE roles (
    role_id BIGSERIAL PRIMARY KEY,
    role_name role_type NOT NULL UNIQUE DEFAULT 'CUSTOMER',
    description TEXT,
    created_by VARCHAR(50),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_by VARCHAR(50),
    updated_at TIMESTAMP WITH TIME ZONE
);
```

-- Employees

```
CREATE TABLE employees (  
    employee_id BIGSERIAL PRIMARY KEY,  
    last_name VARCHAR(50) NOT NULL,  
    first_name VARCHAR(50) NOT NULL,  
    username VARCHAR(50) NOT NULL UNIQUE,  
    address TEXT,  
    phone VARCHAR(20),  
    email VARCHAR(100) NOT NULL UNIQUE,  
    position VARCHAR(50) NOT NULL,  
    salary NUMERIC(10,2) DEFAULT 0,  
    hire_date DATE,  
    created_by VARCHAR(50),  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
    updated_by VARCHAR(50),  
    updated_at TIMESTAMP WITH TIME ZONE  
);
```

-- ... (autres tables selon l'annexe complète)

C. Configuration Spring Boot (application.properties)

properties

```
spring.application.name=gestion_snack  
  
# Configuration de la base de données PostgreSQL  
spring.datasource.url=jdbc:postgresql://localhost:5432/gestion_snack?stringtyp  
e=unspecified  
spring.datasource.username=postgres  
spring.datasource.password=1234  
spring.datasource.driver-class-name=org.postgresql.Driver
```

```

# Configuration JPA/Hibernate
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.properties.hibernate.use_sql_comments=true

# Configuration du pool de connexions
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.minimum-idle=5
spring.datasource.hikari.connection-timeout=20000

# Configuration Swagger/OpenAPI
springdoc.api-docs.path=/api-docs
springdoc.swagger-ui.path=/swagger-ui.html
springdoc.swagger-ui.operationsSorter=method
springdoc.swagger-ui.tagsSorter=alpha

# Configuration du serveur
server.port=8080
server.servlet.context-path=/

# Logging
logging.level.com.joel.gestion_snack=DEBUG
logging.level.org.springframework.web=INFO
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE

```

D. Dépôt Git et Documentation

Dépôt GitHub : [kapello1/gestion-snack](https://github.com/kapello1/gestion-snack): Application de gestion de snack (optionnel)

Documentation API : Swagger UI disponible sur /swagger-ui.html

Postman Collection : Collection exportée dans le dossier /docs

README : Instructions complètes dans le fichier README.md

E. Références Techniques

- ☺ Documentation Spring Boot : <https://spring.io/projects/spring-boot>
- ☺ Documentation PostgreSQL : <https://www.postgresql.org/docs/>
- ☺ Guide React : <https://reactjs.org/docs/getting-started.html>

- 😊 OWASP Security Cheatsheets : <https://cheatsheetseries.owasp.org/>
- 😊 JWT Introduction : <https://jwt.io/introduction>
- 😊 site pour générer le QR Code en donnant le lien: <https://genqrcode.com/>